
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON VAULT - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-VIII-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-VIII.bin      3eb95ef74dcc11bc5a9b77b48645390f17d969c1832a3a7f765ebad7ebd0e120
ACT-VIII.uf2      b653b2b3c1542caf82a7c03d26dd290dc8859882780c2dfdf799d2d69872238d
ACT-VIII_fixed.bin 59411f4bf07809ada695315d75f43c47b7a6efa7c055c8c46841cc6ae0caa234
ACT-VIII_fixed.uf2 4443ce43225991d358e20ab409310209427da83977f6bdf59135a13701ba4a2b
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-VIII.bin` and `ACT-VIII_fixed.bin` SHA-256 values. Both `.bin` images are 50,860 bytes and both `.uf2` images are 102,400 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Vent lock	<code>implant_init</code> (inlined lock gate)	<code>0xA33B</code>	<code>0x1000A33B</code>	<code>0xBB</code>	<code>0xB3</code>
2 LCD mask	<code>implant_mask_active</code>	<code>0xA311</code>	<code>0x1000A311</code>	<code>0xB9</code>	<code>0xB1</code>
3 Lock marker	<code>implant_init</code> (inlined <code>implant_infect</code>)	<code>0xA353</code>	<code>0x1000A353</code>	<code>0xB9</code>	<code>0xB1</code>
4 Vent command authorization	<code>control_handle_frame</code>	<code>0x7595</code>	<code>0x10007595</code>	<code>0xB9</code>	<code>0xB1</code>

Four defects, four changed bytes in four instructions. The disassembly in `ACT-VIII-main-disasm.txt` is taken from the corrected image, so it shows the correct branch encodings.

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-VIII.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronVault_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` ($VA = offset + 0x10000000$).

Vector Table Decoding. First 32 bytes of `ACT-VIII.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	0x10000000
Initial SP	0x20082000
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-VIII-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f902     bl      10006448 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9ff     bl      10006648 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>
1000024c:    4803          ldr     r0, [pc, #12] @ (1000025c <main+0x28>)
1000024e:    f003 fad1     bl      100037f4 <__wrap_puts>
10000252:    2001          movs    r0, #1
10000254:    bd08          pop     {r3, pc}
10000256:    bf00          nop
10000258:    1000ab40     .word   0x1000ab40
1000025c:    1000ab48     .word   0x1000ab48

```

Element	Address
main	0x10000234
monitor_init	0x10006448
monitor_step	0x10006648

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / vault state machine	monitor_init	0x10006448
Monitor / vault state machine	monitor_step	0x10006648
Control (sealed vent path)	control_handle_frame	0x10007528
Damper (actuator)	damper_init	0x100075D4
Damper (actuator)	damper_apply_command	0x100075EC
Damper (actuator)	damper_tick	0x10007620
Damper (actuator)	damper_fail_safe	0x10007660

Module	Anchor function	Address
Vault authorization	<code>vault_auth_apply</code>	<code>0x100076D8</code>
Implant	<code>implant_init</code>	<code>0x1000A320</code>
Implant	<code>implant_tick</code>	<code>0x1000A3B4</code>
Implant	<code>implant_lock_active</code>	<code>0x1000A2FC</code>
Implant	<code>implant_mask_active</code>	<code>0x1000A308</code>
Implant	<code>implant_marker_set</code>	<code>0x1000A2E8</code>
Radio	<code>radio_init</code>	<code>0x1000A444</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007980</code>
Tower light	<code>status_led_show</code>	<code>0x1000A77C</code>

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronVault_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>
[DOCUMENT] main and the vent monitor state machine (monitor_step) addresses identified	1	<code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x10006648</code>
[DOCUMENT] Module map identifies the damper, control, vault_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary` , `ARM Cortex 32 little endian default` , base `0x10000000` , and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C` .
- Always say `reset handler` , never `reset pointer` .
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. `implant_infect` is inlined into `implant_init` and has no standalone symbol.

Task 2: Bug #1 The Vent Lock (20 points)

Solution

Locate the gate. `implant_init` starts at `0x1000A320` and the inlined lock gate is at file offset `0xA33B` (VA `0x1000A33B`). The corrected image is:

```

1000a320 <implant_init>:
1000a320: 2300      movs    r3, #0
1000a322: b530      push    {r4, r5, lr}
1000a324: 481b      ldr     r0, [pc, #108] @ (1000a394 <implant_init+0x74>)
1000a326: 4c1c      ldr     r4, [pc, #112] @ (1000a398 <implant_init+0x78>)
1000a328: 491c      ldr     r1, [pc, #112] @ (1000a39c <implant_init+0x7c>)
1000a32a: 4a1d      ldr     r2, [pc, #116] @ (1000a3a0 <implant_init+0x80>)
1000a32c: 4d1d      ldr     r5, [pc, #116] @ (1000a3a4 <implant_init+0x84>)
1000a32e: 7800      ldrb    r0, [r0, #0]
1000a330: b0c1      sub     sp, #260 @ 0x104
1000a332: 602b      str     r3, [r5, #0]
1000a334: 6023      str     r3, [r4, #0]
1000a336: 700b      strb    r3, [r1, #0]
1000a338: 7013      strb    r3, [r2, #0]
1000a33a: b340      cbz     r0, 1000a38e <implant_init+0x6e>
1000a33c: f04f 23e0  mov.w   r3, #3758153728 @ 0xe000e000
1000a340: f8d3 3df0  ldr.w   r3, [r3, #3568] @ 0xdf0
1000a344: 079b      lsls    r3, r3, #30
1000a346: d122      bne.n   1000a38e <implant_init+0x6e>
1000a348: 2301      movs    r3, #1
1000a34a: 4817      ldr     r0, [pc, #92] @ (1000a3a8 <implant_init+0x88>)
1000a34c: 700b      strb    r3, [r1, #0]
1000a34e: 7800      ldrb    r0, [r0, #0]
1000a350: 7013      strb    r3, [r2, #0]
1000a352: b1e0      cbz     r0, 1000a38e <implant_init+0x6e>
1000a354: 4b15      ldr     r3, [pc, #84] @ (1000a3ac <implant_init+0x8c>)
1000a356: 781b      ldrb    r3, [r3, #0]
1000a358: 2b4c      cmp     r3, #76 @ 0x4c
1000a35a: d018      beq.n   1000a38e <implant_init+0x6e>
1000a35c: f3ef 8410  mrs     r4, PRIMASK
1000a360: b672      cpsid   i
1000a362: 22ff      movs    r2, #255 @ 0xff
1000a364: f10d 0001  add.w   r0, sp, #1
1000a368: 4611      mov     r1, r2
1000a36a: f000 fa67  bl      1000a83c <memset>
1000a36e: 234c      movs    r3, #76 @ 0x4c
1000a370: f44f 5180  mov.w   r1, #4096 @ 0x1000
1000a374: 480e      ldr     r0, [pc, #56] @ (1000a3b0 <implant_init+0x90>)
1000a376: f88d 3000  strb.w  r3, [sp]
1000a37a: f000 fbad  bl      1000aad8 <__flash_range_erase_veneer>
1000a37e: f44f 7280  mov.w   r2, #256 @ 0x100
1000a382: 4669      mov     r1, sp
1000a384: 480a      ldr     r0, [pc, #40] @ (1000a3b0 <implant_init+0x90>)
1000a386: f000 fb8b  bl      1000aaa0 <__flash_range_program_veneer>
1000a38a: f384 8810  msr     PRIMASK, r4
1000a38e: b041      add     sp, #260 @ 0x104
1000a390: bd30      pop     {r4, r5, pc}
1000a392: bf00      nop
1000a394: 20013cf2 .word   0x20013cf2
1000a398: 200136f4 .word   0x200136f4
1000a39c: 20013cf1 .word   0x20013cf1
1000a3a0: 20013cf3 .word   0x20013cf3
1000a3a4: 200136f0 .word   0x200136f0
1000a3a8: 20013cf4 .word   0x20013cf4
1000a3ac: 103ff000 .word   0x103ff000
1000a3b0: 003ff000 .word   0x003ff000

```

Instruction decode. `ldr r0, [pc, #108]` loads the lock gate address `0x20013CF2` (literal at `0x1000A394`), and `ldrb r0, [r0, #0]` reads the gate into `r0` at `0x1000A32E`. `implant_reset_state` runs first: `str r3, [r5, #0]` clears the lock count at `0x200136F0`, `str r3, [r4, #0]` clears the tick counter at `0x200136F4`, `strb r3, [r1, #0]` clears the active flag at `0x20013CF1`, and `strb r3, [r2, #0]` clears the locked flag at `0x20013CF3`. The branch at `0x1000A33A` decides whether the locker may arm. The correct code does nothing when the lock gate is clear, so the branch at `0x1000A33A` must be `cbz (0xB3)` to the `0x1000A38E` return. When the gate is set, the path tests the CoreDebug anti-debug, sets the active and locked flags, and then falls into the marker gate at `0x1000A352`. The condition byte is the high byte at `0x1000A33B`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A33B</code>	<code>0xA33B</code>	<code>0xBB</code>	<code>cbnz r0, 0x1000A38E</code>	<code>0xB3</code>	<code>cbz r0, 0x1000A38E</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA33B</code>	<code>0x1000A33B</code>	<code>40 BB</code>	<code>40 B3</code>

Why the vent is held shut. Under the compromised `cbnz`, the gate is inverted: the fall-through lock path is taken when the lock gate is clear, so boot arms the locker even though the design left the gate clear. The fall-through sets the active flag at `0x20013CF1` and the locked flag at `0x20013CF3`. `implant_lock_active` then returns true, `monitor_open_target` returns false, and the damper is driven closed regardless of the guarded vent state. The yellow LOCKED lamp is lit and the LCD is told the vent is maintenance locked. A perfectly valid, correctly authenticated open command can arrive and the vent still stays shut, because the locker sits beside the sealed command path and overrides its output. After the patch, `cbz` returns while the gate is clear, so the lock is never armed and the vent stays under authorized control. The lesson is that availability is a policy control: no cryptographic control on the envelope can see or stop a local module that decides not to move the damper.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the vent lock gate at <code>0x1000A33B</code>	5	Address and function (<code>implant_init</code> , inlined lock gate) identified
[DOCUMENT] Documented the forced close that holds the vent shut and lights LOCKED	5	Lock gate <code>0x20013CF2</code> , forced closed damper, yellow LOCKED lamp
[DOCUMENT & PATCH] Patched <code>0xBB</code> to <code>0xB3</code> so the vent lock is not armed	7	Byte <code>0xBB</code> changed to <code>0xB3</code>
[DOCUMENT] Explained why the lock forces the vent closed regardless of the authorized state	3	Local override beside the authenticated command path, availability as a policy control

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA33B`; the correct halfword is `b340` for `cbz` and the compromised halfword is `bb40`, so the on-disk bytes are `40 B3` for the fix and `40 BB` for the compromise.
- `cbz` branches when the register is zero (the gate is clear); `cbnz` branches when it is non-zero. The register holds the lock gate, so the semantics are "do not lock while the gate is clear".
- The lock gate is at `0x20013CF2`. The active flag is at `0x20013CF1`, the locked flag at `0x20013CF3`, the tick counter at `0x200136F4`, and the lock count at `0x200136F0`. `implant_lock_active` reads the locked latch at

0x20013CF3 (literal at 0x1000A304).

- The magic release token is VENT_IMPLANT_RELEASE_MAGIC (VAULT-RELEASE-2026) at VENT_IMPLANT_RELEASE_MAGIC_LEN (18) bytes. The release path has no firmware caller, so it is dead-stripped from the shipped image (the token string is absent from ACT-VIII.bin) and the token cannot release the vent; a wrong token, a null pointer, or an attached probe likewise leaves it locked.
- Full credit requires both the byte change and a correct statement of the lesson: the lock is a local condition, not a cipher break, and a device that withholds its own function is an availability failure.

Task 3: Bug #2 The LCD Mask (20 points)

Solution

Locate the gate. implant_mask_active starts at 0x1000A308 and the mask gate is at file offset 0xA311 (VA 0x1000A311). The corrected image is:

```

1000a308 <implant_mask_active>:
1000a308:      4b03          ldr     r3, [pc, #12]    @ (1000a318 <implant_mask_active+0x10>)
1000a30a:      781b          ldrb    r3, [r3, #0]
1000a30c:      f003 00ff    and.w   r0, r3, #255    @ 0xff
1000a310:      b10b          cbz     r3, 1000a316 <implant_mask_active+0xe>
1000a312:      4b02          ldr     r3, [pc, #8]      @ (1000a31c <implant_mask_active+0x14>)
1000a314:      7818          ldrb    r0, [r3, #0]
1000a316:      4770          bx      lr
1000a318:      20013cf5      .word   0x20013cf5
1000a31c:      20013cf3      .word   0x20013cf3

```

Instruction decode. ldr r3, [pc, #12] loads the mask gate address 0x20013CF5 (literal at 0x1000A318), and ldrb r3, [r3, #0] reads the gate. and.w r0, r3, #255 stages the gate value as the return value. The branch at 0x1000A310 decides whether the mask may be reported. The correct code returns false when the mask gate is clear, so the branch at 0x1000A310 must be cbz (0xB1) to the 0x1000A316 return, where r0 still holds zero. When the gate is set, ldr r3, [pc, #8] loads the locked latch at 0x20013CF3 (literal at 0x1000A31C) and returns it. monitor_state_text checks implant_mask_active first and returns MAINT while it is true, so the LCD renders ST:MAINT while the vent is held closed. The condition byte is the high byte at 0x1000A311 .

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A311	0xA311	0xB9	cbnz r3, 0x1000A316	0xB1	cbz r3, 0x1000A316

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA311	0x1000A311	0B B9	0B B1

Why the readout no longer lies. Under the compromised cbnz , the mask gate is inverted: the fall-through mask path is taken when the gate is clear, so implant_mask_active returns the locked latch whenever the vent is locked. monitor_state_text returns MAINT before it ever reaches the real state, so the LCD line renders ST:MAINT and the operator is told the shutdown is routine maintenance. After the patch, cbz returns false while the gate is clear, so the true state is rendered and the mask is never applied. A device that misreports its own state is an availability failure with no visible symptom except the one it is allowed to show, which is why the truth on the display is a security control and not a cosmetic detail.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the mask gate at 0x1000A311	5	Address and function (<code>implant_mask_active</code>) identified
[DOCUMENT] Documented the ST:MAINT maintenance mask that hides the true vent state	5	Mask gate <code>0x20013CF5</code> , <code>monitor_state_text</code> returns <code>MAINT</code> while locked
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so the true state is never masked	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why a masked state hides an availability failure	3	The readout is part of the attack surface and the truth is a control

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA311` ; the correct halfword is `b10b` for `cbz` and the compromised halfword is `b90b` , so the on-disk bytes are `0B B1` for the fix and `0B B9` for the compromise.
- The mask gate is at `0x20013CF5` , and the value returned when the mask is active is the locked latch at `0x20013CF3` , not the gate. This is exactly the behavior the compromised image wants: mask only while locked.
- `monitor_state_text` returns `MAINT` and the LCD line is formatted by `monitor_status_render` as `ST:MAINT` while `implant_mask_active` is true.
- Full credit requires both the byte change and a correct statement of the lesson: hiding the state is part of the lockout, and the operator readout is an attack surface.

Task 4: Bug #3 The Lock Marker (20 points)

Solution

Locate the gate. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A320`). The marker gate is at file offset `0xA353` (VA `0x1000A353`). The corrected image is:

```

1000a348:      2301      movs    r3, #1
1000a34a:      4817      ldr     r0, [pc, #92] @ (1000a3a8 <implant_init+0x88>)
1000a34c:      700b      strb   r3, [r1, #0]
1000a34e:      7800      ldrb   r0, [r0, #0]
1000a350:      7013      strb   r3, [r2, #0]
1000a352:      b1e0      cbz    r0, 1000a38e <implant_init+0x6e>
1000a354:      4b15      ldr     r3, [pc, #84] @ (1000a3ac <implant_init+0x8c>)
1000a356:      781b      ldrb   r3, [r3, #0]
1000a358:      2b4c      cmp    r3, #76 @ 0x4c
1000a35a:      d018      beq.n  1000a38e <implant_init+0x6e>
1000a35c:      f3ef 8410  mrs    r4, PRIMASK
1000a360:      b672      cpsid  i
1000a362:      22ff      movs   r2, #255 @ 0xff
1000a364:      f10d 0001  add.w  r0, sp, #1
1000a368:      4611      mov    r1, r2
1000a36a:      f000 fa67  bl     1000a83c <memset>
1000a36e:      234c      movs   r3, #76 @ 0x4c
1000a370:      f44f 5180  mov.w  r1, #4096 @ 0x1000
1000a374:      480e      ldr     r0, [pc, #56] @ (1000a3b0 <implant_init+0x90>)
1000a376:      f88d 3000  strb.w r3, [sp]
1000a37a:      f000 fbad  bl     1000aad8 <__flash_range_erase_veneer>
1000a37e:      f44f 7280  mov.w  r2, #256 @ 0x100
1000a382:      4669      mov    r1, sp
1000a384:      480a      ldr     r0, [pc, #40] @ (1000a3b0 <implant_init+0x90>)
1000a386:      f000 fb8b  bl     1000aaa0 <__flash_range_program_veneer>
1000a38a:      f384 8810  msr    PRIMASK, r4
1000a38e:      b041      add    sp, #260 @ 0x104
1000a390:      bd30      pop    {r4, r5, pc}
1000a392:      bf00      nop
1000a394:      20013cf2 .word  0x20013cf2
1000a398:      200136f4 .word  0x200136f4
1000a39c:      20013cf1 .word  0x20013cf1
1000a3a0:      20013cf3 .word  0x20013cf3
1000a3a4:      200136f0 .word  0x200136f0
1000a3a8:      20013cf4 .word  0x20013cf4
1000a3ac:      103ff000 .word  0x103ff000
1000a3b0:      003ff000 .word  0x003ff000

```

Instruction decode. After the lock gate is armed, `ldr r0, [pc, #92]` loads the marker gate address `0x20013CF4` (literal at `0x1000A3A8`) and `ldrb r0, [r0, #0]` reads it at `0x1000A34E`. The branch at `0x1000A352` decides whether the marker may be written. The correct code writes no marker when the gate is clear, so the branch at `0x1000A352` must be `cbz (0xB1)` to the `0x1000A38E` return. When the gate is set, the reserved sector address `0x103FF000` is loaded (literal at `0x1000A3AC`) and the present marker is checked with `cmp r3, #76 (0x4C)`. If the marker is absent, the Pico SDK flash sequence runs: the marker byte `0x4C` is staged at `0x1000A34C / 0x1000A350`, then `flash_range_erase` at `0x1000A37A` and `flash_range_program` at `0x1000A386` program the sector through the veneers at `0x1000AAD8` and `0x1000AAA0`. The condition byte is the high byte at `0x1000A353`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A353	0xA353	0xB9	cbnz r0, 0x1000A38E	0xB1	cbz r0, 0x1000A38E

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA353	0x1000A353	E0 B9	E0 B1

The anti-debug obstacle. The locker reads `CoreDebug_DHCSR` at `0xE000EDF0` and returns early while a probe is attached. In `implant_init`:

```

1000a33c:    f04f 23e0    mov.w    r3, #3758153728 @ 0xe000e000
1000a340:    f8d3 3df0    ldr.w    r3, [r3, #3568] @ 0xdf0
1000a344:    079b        lsls     r3, r3, #30
1000a346:    d122        bne.n    1000a38e <implant_init+0x6e>

```

The same register is read in `implant_tick`:

```

1000a3b4 <implant_tick>:
1000a3b4:    f04f 21e0    mov.w    r1, #3758153728 @ 0xe000e000
1000a3b8:    4a1c        ldr      r2, [pc, #112] @ (1000a42c <implant_tick+0x78>)
1000a3ba:    6813        ldr      r3, [r2, #0]
1000a3bc:    3301        adds     r3, #1
1000a3be:    6013        str      r3, [r2, #0]
1000a3c0:    f8d1 2df0    ldr.w    r2, [r1, #3568] @ 0xdf0
1000a3c4:    0792        lsls     r2, r2, #30
1000a3c6:    d12c        bne.n    1000a422 <implant_tick+0x6e>
1000a3c8:    4a19        ldr      r2, [pc, #100] @ (1000a430 <implant_tick+0x7c>)
1000a3ca:    7812        ldrb     r2, [r2, #0]
1000a3cc:    b342        cbz      r2, 1000a420 <implant_tick+0x6c>
1000a3ce:    079b        lsls     r3, r3, #30
1000a3d0:    d126        bne.n    1000a420 <implant_tick+0x6c>
1000a3d2:    2101        movs     r1, #1
1000a3d4:    4b17        ldr      r3, [pc, #92] @ (1000a434 <implant_tick+0x80>)
1000a3d6:    4a18        ldr      r2, [pc, #96] @ (1000a438 <implant_tick+0x84>)
1000a3d8:    781b        ldrb     r3, [r3, #0]
1000a3da:    7011        strb     r1, [r2, #0]
1000a3dc:    b303        cbz      r3, 1000a420 <implant_tick+0x6c>
1000a3de:    4b17        ldr      r3, [pc, #92] @ (1000a43c <implant_tick+0x88>)
1000a3e0:    781b        ldrb     r3, [r3, #0]
1000a3e2:    2b4c        cmp      r3, #76 @ 0x4c
1000a3e4:    d01c        beq.n    1000a420 <implant_tick+0x6c>
1000a3e6:    b510        push     {r4, lr}
1000a3e8:    b0c0        sub      sp, #256 @ 0x100
1000a3ea:    f3ef 8410    mrs      r4, PRIMASK
1000a3ee:    b672        cpsid    i
1000a3f0:    22ff        movs     r2, #255 @ 0xff
1000a3f2:    eb0d 0001    add.w    r0, sp, r1
1000a3f6:    4611        mov      r1, r2
1000a3f8:    f000 fa20    bl       1000a83c <memset>
1000a3fc:    234c        movs     r3, #76 @ 0x4c
1000a3fe:    f44f 5180    mov.w    r1, #4096 @ 0x1000
1000a402:    480f        ldr      r0, [pc, #60] @ (1000a440 <implant_tick+0x8c>)
1000a404:    f88d 3000    strb.w   r3, [sp]
1000a408:    f000 fb66    bl       1000aad8 <__flash_range_erase_veneer>
1000a40c:    f44f 7280    mov.w    r2, #256 @ 0x100
1000a410:    4669        mov      r1, sp
1000a412:    480b        ldr      r0, [pc, #44] @ (1000a440 <implant_tick+0x8c>)
1000a414:    f000 fb44    bl       1000aaa0 <__flash_range_program_veneer>
1000a418:    f384 8810    msr      PRIMASK, r4
1000a41c:    b040        add      sp, #256 @ 0x100
1000a41e:    bd10        pop      {r4, pc}
1000a420:    4770        bx       lr
1000a422:    2200        movs     r2, #0
1000a424:    4b04        ldr      r3, [pc, #16] @ (1000a438 <implant_tick+0x84>)
1000a426:    701a        strb     r2, [r3, #0]
1000a428:    4770        bx       lr
1000a42a:    bf00        nop
1000a42c:    200136f4    .word    0x200136f4
1000a430:    20013cf1    .word    0x20013cf1
1000a434:    20013cf4    .word    0x20013cf4
1000a438:    20013cf3    .word    0x20013cf3
1000a43c:    103ff000    .word    0x103ff000
1000a440:    003ff000    .word    0x003ff000

```

The shift `lsls r3, r3, #30` keeps bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) in the carry and sign positions, and the `bne` returns early when either bit is set. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```
arm-none-eabi-gdb ACT-VIII.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A38A
(gdb) continue
(gdb) x/4xb 0x103FF000
```

To observe the boot write on the compromised image, break after the flash program at `0x1000A38A` (`msr PRIMASK, r4`) in `implant_init`, then read the reserved sector at `0x103FF000` and confirm the first byte is `4C`. To observe the tick re-assertion, clear the debug bits (or patch the `ldr.w` at `0x1000A3C0` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `cbnz`, the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes `0x4C` to `0x103FF000`. After the patch, `cbz` returns while the gate is clear, so the flash erase and program at `0x1000A37A` and `0x1000A386` are never reached and the sector stays blank. The marker is the durable state that re-arms the lock on every later boot, and the reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the lock marker gate at <code>0x1000A353</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no lock marker is programmed to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the lock marker byte <code>0x4C</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA353`; the correct halfword is `b1e0` for `cbz` and the compromised halfword is `b9e0`, so the on-disk bytes are `E0 B1` for the fix and `E0 B9` for the compromise.
- The marker byte is `VENT_IMPLANT_LOCK_MARKER` (`0x4C`), the reserved sector is `VENT_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the marker gate is at `0x20013CF4`. The write uses the real API, `flash_range_erase` and

`flash_range_program`, through the veneers at `0x1000AAD8` and `0x1000AAA0`.

- The `DHCSR` address is `VENT_IMPLANT_DHCSR_ADDR` (`0xE00EDF0`); bit 0 is `VENT_IMPLANT_DHCSR_DEBUGEN` (`0x00000001`) and bit 1 is `VENT_IMPLANT_DHCSR_HALT` (`0x00000002`). The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA353` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.
- The source can clear the marker on a successful token release, but the shipped image dead-strips that path, so the documented fix is the patch plus a reserved-sector erase, not the token.

Task 5: Bug #4 The Vent Command Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x10007528`) the authorization branch is at file offset `0x7595` (VA `0x10007595`). The corrected image is:

```

10007528 <control_handle_frame>:
10007528: 2107      movs    r1, #7
1000752a: b530      push   {r4, r5, lr}
1000752c: 4b1e      ldr     r3, [pc, #120] @ (100075a8 <control_handle_frame+0x80>)
1000752e: b097      sub     sp, #92 @ 0x5c
10007530: 781a      ldrb    r2, [r3, #0]
10007532: f88d 1018 strb.w   r1, [sp, #24]
10007536: 2a00      cmp     r2, #0
10007538: d033      beq.n   100075a2 <control_handle_frame+0x7a>
1000753a: 2800      cmp     r0, #0
1000753c: d031      beq.n   100075a2 <control_handle_frame+0x7a>
1000753e: 2430      movs    r4, #48 @ 0x30
10007540: a905      add     r1, sp, #20
10007542: aa0a      add     r2, sp, #40 @ 0x28
10007544: 4603      mov     r3, r0
10007546: 9102      str     r1, [sp, #8]
10007548: 9200      str     r2, [sp, #0]
1000754a: 4818      ldr     r0, [pc, #96] @ (100075ac <control_handle_frame+0x84>)
1000754c: 2201      movs    r2, #1
1000754e: a906      add     r1, sp, #24
10007550: 9401      str     r4, [sp, #4]
10007552: f000 fa15 bl       10007980 <envelope_open_hex>
10007556: b320      cbz     r0, 100075a2 <control_handle_frame+0x7a>
10007558: 9b05      ldr     r3, [sp, #20]
1000755a: 2b16      cmp     r3, #22
1000755c: d921      bls.n   100075a2 <control_handle_frame+0x7a>
1000755e: f89d 402c ldrb.w   r4, [sp, #44] @ 0x2c
10007562: f8bd 302d ldrh.w   r3, [sp, #45] @ 0x2d
10007566: 1e62      subs    r2, r4, #1
10007568: 2a02      cmp     r2, #2
1000756a: b21d      sxth    r5, r3
1000756c: d819      bhi.n   100075a2 <control_handle_frame+0x7a>
1000756e: 2b10      cmp     r3, #16
10007570: d817      bhi.n   100075a2 <control_handle_frame+0x7a>
10007572: f8dd 002f ldr.w    r0, [sp, #47] @ 0x2f
10007576: f8dd 1033 ldr.w    r1, [sp, #51] @ 0x33
1000757a: f8dd 2037 ldr.w    r2, [sp, #55] @ 0x37
1000757e: f8dd 303b ldr.w    r3, [sp, #59] @ 0x3b
10007582: f10d 0c18 add.w    ip, sp, #24
10007586: e8ac 000f stmia.w  ip!, {r0, r1, r2, r3}
1000758a: 990a      ldr     r1, [sp, #40] @ 0x28
1000758c: 4808      ldr     r0, [pc, #32] @ (100075b0 <control_handle_frame+0x88>)
1000758e: aa06      add     r2, sp, #24
10007590: f000 f8a2 bl       100076d8 <vault_auth_apply>
10007594: b128      cbz     r0, 100075a2 <control_handle_frame+0x7a>
10007596: 4a07      ldr     r2, [pc, #28] @ (100075b4 <control_handle_frame+0x8c>)
10007598: 4b07      ldr     r3, [pc, #28] @ (100075b8 <control_handle_frame+0x90>)
1000759a: 7014      strb    r4, [r2, #0]
1000759c: 801d      strh    r5, [r3, #0]
1000759e: b017      add     sp, #92 @ 0x5c
100075a0: bd30      pop     {r4, r5, pc}
100075a2: 2000      movs    r0, #0
100075a4: b017      add     sp, #92 @ 0x5c
100075a6: bd30      pop     {r4, r5, pc}
100075a8: 20013ced .word   0x20013ced
100075ac: 200136c8 .word   0x200136c8
100075b0: 200136ac .word   0x200136ac
100075b4: 20013cec .word   0x20013cec
100075b8: 20013ce2 .word   0x20013ce2

```

Instruction decode. The sealed frame is opened under the field key by `envelope_open_hex` at `0x10007552`, and a malformed or too-short body is rejected at `0x10007556` and `0x1000755c`. The command byte is checked against the guarded vent set by `subs r2, r4, #1` and `cmp r2, #2` at `0x10007566` and `0x10007568`, and the zone is checked against the `0` to `16` band by `cmp r3, #16` at `0x1000756E`. `vault_auth_apply` at `0x10007590` enforces the session-scoped anti-replay window and checks the command tag against the candidate record it would produce, returning its authorization verdict in `r0`. The branch at `0x10007594` decides whether the command

may reach the applied command and zone. The correct code rejects a failed or replayed authorization, so the branch at `0x10007594` must be `cbz (0xB1)` to the `0x100075A2` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013CEC` and the zone at `0x20013CE2`. The condition byte is the high byte at `0x10007595`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x10007595</code>	<code>0x7595</code>	<code>0xB9</code>	<code>cbnz r0, 0x100075A2</code>	<code>0xB1</code>	<code>cbz r0, 0x100075A2</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x7595</code>	<code>0x10007595</code>	28 B9	28 B1

Why the command now requires authorization. Under the compromised `cbnz`, the verdict is inverted: a failed or replayed authorization falls through to the stores at `0x1000759A`, while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x100075A2`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the command byte and zone are applied. That replay rejection is session-scoped: the window lives in `.bss` and `vault_auth_init` zeroes `last_seq` at boot, so a captured frame with `seq >= 1` still replays after a power cycle. The patch restores authorization, not durable anti-replay. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `VENT_COMMAND_OPEN (0x01)`, `VENT_COMMAND_CLOSE (0x02)`, and `VENT_COMMAND_PURGE (0x03)`, and the zone is checked against the band `0` to `16`. This is the defect that is a policy seam rather than implant behavior, and it is the one a defender would fix first in production.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the vent command authorization branch at <code>0x10007595</code>	5	Address and function (<code>control_handle_frame</code>) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why an unauthenticated or replayed vent command must be rejected	3	The applied command must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x7595`; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928`, so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `vault_auth_apply` (starts at `0x100076D8`) performs the monotonic anti-replay check and the command-tag check, so this branch is the verdict for authentication. Be precise about the limits: the window lives in `.bss` and `vault_auth_init` sets `last_seq = 0` at boot, so it is monotonic only within a power session; the stored state tag is a deterministic MAC over (`GRANT`, `seq`, `last_seq=seq`) and `vault_auth_state_ok` has no firmware caller, so it never gates apply. A captured frame with `seq >= 1` still replays after a power cycle.

- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the vent command path is correct. Only the verdict seam was broken.
- The applied command is at `0x20013CEC`, and the applied zone is at `0x20013CE2`. The control ready gate is at `0x20013CED`, and the authorization record is at `0x200136AC`.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, `File -> Export Program...`, choose `Binary Format`, and save as `ACT-VIII_fixed.bin`. The shipped image is 50,860 bytes.

Convert.

```
python uf2conv.py ACT-VIII_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VIII_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-VIII is 102,400 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-VIII_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- the vent is no longer forced closed and the yellow LOCKED lamp is not lit by the locker;
- the LCD renders the true state instead of `ST:MAINT`;
- an unauthenticated command and a replayed captured command are rejected before the command and zone are applied within a power session (a captured frame with `seq >= 1` still replays across a power cycle, because the window resets at boot);
- a legitimate authorized command still applies, and the manual purge request, the maintenance remote, and the fail-open policy still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Vent Lock	<code>0xA33B</code>	<code>0x1000A33B</code>	<code>BB</code>	<code>B3</code>
2	The LCD Mask	<code>0xA311</code>	<code>0x1000A311</code>	<code>B9</code>	<code>B1</code>
3	The Lock Marker	<code>0xA353</code>	<code>0x1000A353</code>	<code>B9</code>	<code>B1</code>

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
4	The Vent Command Authorization	0x7595	0x10007595	B9	B1

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Vent Lock	A local condition holds a cooling damper shut while every light says normal, so a server hall heats while the controller refuses to act. An availability attack needs no cipher break.
The LCD Mask	A device misreports its own state as maintenance, so an operator readout becomes a means of concealment rather than a means of detection.
The Lock Marker	A payload writes a durable marker to a reserved sector, so the state that re-arms the lockout survives remediation and a reflash.
The Vent Command Authorization	An inverted verdict lets an unauthenticated or replayed command change a physical damper decision and zone, so authorization is defeated without breaking authentication.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported ACT-VIII_fixed.bin from Ghidra	2	Valid patched binary
[PATCH] Converted to ACT-VIII_fixed.uf2 with the correct base and family	2	--base 0x10000000 --family 0xe48bff59
[DOCUMENT] scripts/verify_ctf.py passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from ACT-VIII.bin in exactly the four bytes in the table; scripts/verify_ctf.py checks this and the SHA-256 values.
- Confirm the UF2 conversion used base 0x10000000 and family 0xe48bff59.
- The shipped image is 50,860 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.
- The complete fix is also a policy: fail open on a lost link, treat a local purge as a request, remove the SANDBOX_ONLY locker code path, and clear the reserved sector. The four byte patches close the shipped seams; the policy closes the class.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 rack temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address 0x27
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Vent damper servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HALL HOT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	LOCKED, 220 to 330 ohm to GND
Green LED	anode	GP18	COOLING OK, 220 to 330 ohm to GND
Manual purge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The Vent Lock	20
Task 3	Bug #2 The LCD Mask	20
Task 4	Bug #3 The Lock Marker	20
Task 5	Bug #4 The Vent Command Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational datacenter network, a building-management system, a cooling plant control system, a public network, a military system, or a third-party device. The locker is benign and confined to the breadboard: it affects only the mock vent and the mock LCD, is neutralized by the documented patch (its release-token path is compiled out of the shipped image), and writes only the reserved sector at `0x103FF000` on the same chip. There is no network, no filesystem, and no host impact.

Common Student Mistakes

- Patching the low byte of the branch at `0xA33A`, `0xA310`, `0xA352`, or `0x7594` instead of the condition byte at `0xA33B`, `0xA311`, `0xA353`, or `0x7595`.
- Reading the vent lock gate backwards and believing the corrected build still forces the vent closed.
- Reading the mask gate backwards and believing the corrected build still renders `ST:MAINT`.
- Searching for a standalone `implant_infect` symbol and missing that it is inlined into `implant_init`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the lock is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two clearing gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting that the fix is also a policy: fail open on a lost link and never let a local request silently bypass authorization.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award partial credit for identifying the availability nature of the lock without connecting it to the fail-open policy, or the policy without the lock.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```

--- ACT-VIII.bin (compromised)
+++ ACT-VIII_fixed.bin (corrected)

```

```

Offset 0x00007595: B9 -> B1 (cbnz r0, 0x100075A2 -> cbz r0, 0x100075A2)
Offset 0x0000A311: B9 -> B1 (cbnz r3, 0x1000A316 -> cbz r3, 0x1000A316)
Offset 0x0000A33B: BB -> B3 (cbnz r0, 0x1000A38E -> cbz r0, 0x1000A38E)
Offset 0x0000A353: B9 -> B1 (cbnz r0, 0x1000A38E -> cbz r0, 0x1000A38E)

```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Vent Lock	0xA33B	0x1000A33B	40 BB	40 B3
2	The LCD Mask	0xA311	0x1000A311	0B B9	0B B1
3	The Lock Marker	0xA353	0x1000A353	E0 B9	E0 B1
4	The Vent Command Authorization	0x7595	0x10007595	28 B9	28 B1

Four defects, four changed bytes in four instructions: the vent lock gate, the display mask gate, the lock marker gate, and the authorization verdict. No other byte in either image differs. The CoreDebug DHCSR anti-debug is present and identical in both images, so it is an analysis obstacle and not a fifth patch.